Completion Frontiers for Long-Lived Agent Tasks

OpenBot

***Abstract*—An agent may finish an answer while child work, decisions, reminders, or accepted corrections remain open. We define a completion frontier for one round of work in a reused context. The method preserves a record of each promise during delivery and closes a round only when two state versions still match a checked candidate. We give a state model, a proof of safe close, and a conditional settlement argument. The proof separates what the host records from whether the model used evidence correctly. A finite executable model checks event orders and distinct failures from early result release and stale close candidates. These checks establish bounded model behavior, not production correctness or model quality. We also specify a paired study at equal total resources, with separate measures for false completion, useful response time, and task success. No model performance gain is claimed.**

***Index Terms*—Agent coordination, Asynchronous tasks, Completion detection, Task budgets**

I. Introduction

A person may change a request while a helper works. A helper may need another helper. Results may arrive after the lead agent ends its answer, and tasks may continue with their old context. The team needs a rule for what is still owed and when one round is over.

We study agents that retain a role across many tasks. Each task has a context containing its input and history. An activation is one run that reads available input and produces an answer or tool action. A delegation assigns work to a child context and returns a receipt before the result is ready. The result later returns as a notice, a message about a protocol event. The parent can thus answer the person without waiting for all child work.

Removing synchronous waits does not prove that work will finish. Tasks can keep asking for help or setting reminders. People can keep adding work. Paused tasks can await decisions forever. A settled team can still give a wrong answer. We separate responsiveness, settlement, and quality.

This paper makes three contributions. First, it gives a round model that preserves the identity of work across reused contexts. Second, it proves a guarded close rule from explicit records of promises and accepted input. Third, it supplies finite executable checks and a study design that separates completion safety from model quality. The finite checks have been run. The matched-resource study with model agents has not.

The research question is narrow: can a host close a round without losing work that it already accepted? The answer depends on an explicit storage contract and complete work records. Nonblocking delegation, message passing, and distributed termination detection are established methods. Our contribution combines these ideas into a round-level contract for open-ended agent services, including late corrections and reuse of context. It is not a new general termination detector.

The focus is organizational coordination: task assignment, results owed, and responsiveness to people. Storage recovery, access control, and outside write safety are assumed services. The executable model is a separate reference model. Passing it does not show that the service implements every transition in this paper.

II. Relation to Earlier Work

The actor model made message passing a basic form of concurrent computation [1]. An actor can respond and create further activity without a shared call stack. Our agent roles may have several active contexts, so a role is not one sequential mailbox. The unit for input order is the context; the unit for settlement is the work round.

Dijkstra and Scholten study termination detection for diffusing computations [2]. The key lesson is that local inactivity does not show global termination. Work can still be owed through message paths. Their protocol tracks obligations created by sent work. Our child records serve a related purpose, but future reminders and human decisions extend the problem beyond a closed computation with process messages.

Chandy and Lamport show how to record a consistent global state while distributed processes continue to run [3]. Messages in transit matter to that state. Our completion frontier follows this concern: a child result must not vanish between the open-child set and the parent's accepted input. We use a per-task argument and a proposed guarded close, not the snapshot algorithm. A fresh user request is also new work, rather than evidence that a previously closed scope was never complete.

AutoGen expresses agent applications through configurable conversations, with agents that can use models, tools, and human input [4]. CAMEL studies role-based cooperation between language-model agents [5]. Our independent variable is narrower: how tasks remain open across activations and how results and revisions cross their completion boundary. Role prompts alone do not define that boundary.

Mattern studies distributed termination detection with message counting in systems that need not deliver messages in first-in, first-out order [6]. That work strengthens the comparison with a simple test that every worker is idle. A stopped worker may still receive a message. Our method applies the same concern to reminders, decision waits, and task extensions, without claiming a new general distributed termination algorithm.

AgentScope provides a multi-agent platform with an actor-based distributed execution framework [7]. Placeholder messages let work continue until a result is needed. Thus nonblocking calls and automatic parallel work are established

design choices. Our proposed addition is an explicit boundary for which user inputs and task obligations one completion report covers, including when a later round reuses a context.

The MAST study separates unawareness of stopping conditions, premature termination, and failures to verify an output [8]. CAMEL also reports repetitive exchanges after apparent task completion and uses a message limit to bound conversations [5]. These findings motivate separate measures for settlement and quality. We compare policies at equal total budgets and do not assume that parallelism, role diversity, or a larger team improves every task.

Herlihy and Wing define linearizability: each operation appears to take effect at one point between its call and return [9]. This is the storage requirement behind our version guard. The guard gives an atomic close relative to input admission for one context. It does not establish a global snapshot of every role, and it does not replace a proof that the storage operations meet that requirement.

TABLE I
THE METHOD BUILDS ON EXISTING COORDINATION IDEAS. THE TABLE STATES THE SCOPE OF THIS PAPER, NOT MISSING FEATURES IN ALL EARLIER SYSTEMS.

Earlier method	Work retained here	Question added here
Diffusing computation	Track work still owed	Which round covers new input?
Distributed snapshot	Keep in-flight work visible	Can admission race with close?
Linearizable object	Give operations one order	Which work facts must close check?
Agent conversations	Roles, tools, and messages	Was accepted evidence given a disposition?

III. Task Model and Scope

An **agent role** is a stable set of instructions and abilities. A **context** is an ordered input queue and history. A context has at most one advancing **activation**, which is a finite run over accepted input. A **round** is one assignment with an identifier, parent context, child context, task, state, limits, and result. Several rounds can reuse one context in order. Several contexts can use one role at once. Reusing a role does not imply sharing task history.

A round is queued before its first activation, running while it can advance, or paused while continuation awaits a decision. A running round may be idle between activations. It is still open if it owes work. Completed, failed, and cancelled are end states. We use **settled** for any end state and **successful** only for work that passes an independent task check. The protocol cannot infer answer quality from silence.

A notice has a stable event identifier, origin round, target round, kind, and body. Kinds include child result, decision result, and reminder. A result notice also names the obligation it resolves and the child's end status. Acceptance assigns an input position. Repeated delivery of the same event identifier and body returns the prior acceptance record. It adds no input and changes no version. Reusing an identifier with a different body is an error. The receiver checks the target round and obligation before admission. This is an abstract receiver requirement, not a claim about a particular transport's delivery guarantees.

All sets below belong to the current round of context c. A settled earlier round has separate records. Let $D(c)$ be its open child obligations, $Q(c)$ its pending decisions, and $R(c)$ its future reminders. Let $I(c)$ be current-round work input without a recorded disposition. A disposition states that an activation used the input or that failure or cancellation explicitly abandoned it. This bookkeeping does not prove that a model understood the input. Let $A(c)$ mean that an activation is advancing and $P(c)$ mean that its round is paused. A candidate close requires

$$|D(c)| + |Q(c)| + |R(c)| + |I(c)| = 0, \tag{1}$$

$$\neg A(c) \quad \text{and} \quad \neg P(c). \tag{2}$$

Completed work also needs a terminal activation record. Failed or cancelled work may instead have an explicit stop record. A completed round requires use dispositions; abandonment forces a failed or cancelled result. Thus a task that never began cannot appear completed. Children carry promised results. Decisions can change the plan. Reminders reserve future activity. Accepted input can change the answer after every child has ended. Input moved from a queue into an activation stays in $I(c)$ until that activation records its disposition. Emptying the queue alone does not make the input disappear.

A context revision v increases on input admission, input disposition, activation changes, pause changes, result changes, and round changes. An obligation revision w increases on any addition, resolution, or cancellation in D, Q, or R. Revisions never decrease or repeat, even when a value changes and later changes back. A finite counter must not wrap within the life of a context. All such mutations and a round close have one atomic ordering point for that context. Each read returns its state block and revision from one atomic snapshot. The two blocks may be read separately. Every writer changes current state and its revision together; stale writes must retry. Close compares both revisions atomically. This abstract storage contract defines what a future implementation must provide.

For liveness claims we add finite work, eventual service, and resolved waits. We do not assume equal task duration, a fixed number of roles, or fixed delegation depth. A service accepting unbounded new work cannot have permanent global termination. We ask whether a particular round settles and whether new work receives an explicit scope.

A. Records, failure model, and trust

A **work record** names either an accepted task input or a promised result. Its identity is stable. For every promised result j, the host keeps at least one of three witnesses: an open obligation, an accepted result input, or a saved disposition. During delivery it may keep two. The sum of open obligations and inputs is therefore a close test, not a count of distinct tasks.

For each admitted input x, exactly one round owns its position. Before settlement it has either no disposition, a use record from an activation, or an explicit abandonment record. A history-only record is allowed only for a notice whose target round was already settled. A normal input must not be

relabeled as history to make close pass. The host checks these records; a model cannot retire obligations by stating that work is complete.

We allow process stops, repeated delivery, delay, and reordering. Saved records survive a process stop. A parent keeps an obligation until result acceptance or explicit cancellation is saved. A child keeps a pending delivery record until the parent confirms acceptance. After a stop, workers retry pending deliveries with the same event identity. Lost acknowledgments can thus cause repeats but not a missing work record. Permanent storage loss and forged control records are outside the model. An indefinitely lost message preserves safety by leaving work open; it prevents settlement.

An activation carries its round ID and an ownership epoch, a number that changes when a new worker takes control. Each activation write checks both under the same ordering point as its write. A stopped or stale worker cannot start a new child, dispose of input, or publish an answer after losing ownership. Notice receipt and explicit control actions use separate host authority. Cancellation first records a stop that blocks new activation work. It then resolves each remaining work record with a reason. Logical cancellation does not imply that an outside action already sent over the network had no effect.

These are assumptions of the protocol, not consequences of asynchronous calls. They state the exact obligations of a storage and execution layer that implements the method. Model errors, false content, and unrecorded work in natural-language promises remain possible even if these assumptions hold.

IV. Coordination Method

A. Give work and return a receipt

The parent writes a clear task, including the needed facts and expected result. A fresh child context receives only the facts included in its assignment. The parent then creates an open delegation and receives its identifier. The parent registers the obligation before dispatching the assignment. Registration and the parent revision change share one ordering point. The receipt shows acceptance and current state. It does not stand in for the child's result. The parent can continue independent work or end its current answer while the delegation remains open.

An assignment should name a testable deliverable and stopping point, such as three failing cases with their inputs. This convention states a finite intended scope but cannot force model compliance.

An agent may assign work to another context with its own role. It may also assign work to another role that delegates back to the first role. The role graph can therefore have cycles while the graph of newly created task instances remains a tree. This distinction explains why a cycle of role names is not by itself a resource deadlock. It also explains why such cycles can generate an endless sequence of fresh tasks.

B. Keep obligations visible

When a child settles, its notice is accepted by the parent before the parent removes the child obligation. During this handoff both records may be visible. That temporary overlap is safe for completion detection. A gap in which neither is visible is unsafe.

The receiver must distinguish acceptance from incorporation. Acceptance means the message has a place in the context input order. Incorporation means a terminal activation has recorded its disposition. For a normal result this records use; for a stopped round it records abandonment with a reason. A child result that merely reaches an inbox cannot justify a finished parent answer. A short receipt to the person may still be appropriate, but it must not claim that all assigned work has been integrated.

The completion check reads obligations before the context state. For a check without a final version guard, this order matters. If it sees an open child, it waits. If the child obligation is absent, its notice must already have been accepted or an explicit cancellation must have removed the obligation. The later context read then sees the input queued, an activation processing it, or a terminal activation after incorporation. Reversing those reads could see an old empty inbox and then the new closed child. With both revisions checked atomically at commit, either read order is safe: a changed block rejects the candidate. We retain the stated order to expose the handoff argument and avoid work on an already open child.

C. Close at an explicit frontier

We propose a **completion frontier**: a recorded boundary through the inputs and obligations covered by one settled round. A frontier contains the round ID, the greatest accepted input position h, both checked revisions, the terminal result record, and the end state. Every input owned by that round at a position at most h must have a disposition. Recording only the largest used position would permit a gap at an earlier input. The closed record therefore certifies a full prefix of the round's accepted inputs, not just its last item. It makes the question precise: completed with respect to which accepted work?

A guarded close atomically checks that the context revision and obligation revision still equal the values read during the candidate check. Input admission and round close use the same ordering boundary. Input that wins that order belongs to the current round and blocks close. Input that arrives after close starts a new round, if its kind permits new work. The close writes the round end state, result, and frontier at that same ordering point. Rechecking only one revision is insufficient: a new child can change obligations without adding user input, while a correction can add input without creating a child.

```
On child settlement:
  accept result notice in parent context
  close the child's open obligation

On a candidate task end:
  read open obligations and revision w
  read context revision v, input, and activation
  if any obligation, input, activation, or pause remains:
    return
  if completing:
    require terminal activation and use dispositions
  if failing or cancelling:
    require terminal activation or explicit stop record
  in one atomic action, require the round and both versions
to match
    require the round to remain open
```

```
    close and record frontier, result, and end state

On later accepted work:
  keep the old round closed
  make or use the next round in the same context
```

The pseudocode defines an ordering contract without a global barrier that forces every agent role to become idle.

D. Extend tasks without erasing earlier results

An extension to open work keeps its round and limits. An extension after settlement creates another round with new limits in the same context. The prior result remains unchanged.

Input admission first checks the current round under the context ordering point. If the round remains open, a task extension joins it. If it has settled, an explicit new task creates the next round and gets a new identifier. The next round records its predecessor, so retained history does not erase the distinction between the two assignments. Wall-clock times do not decide membership.

A duplicate result is not a new request. A late notice whose target round has settled is retained as history, with a history-only disposition, and cannot reopen work. A notice from a settled child still targets its open parent normally. A new round requires an explicit task input or extension. A reminder or decision associated with open work must use the same notice-before-removal handoff as a child result. Failure or cancellation stops further child creation, resolves or cancels remaining obligations, and records dispositions before the round settles. The pause blocks model activations, not notice receipt or control actions. An allow decision releases the pause; a deny decision clears the pause and starts cancellation under a stop record. Thus the continuation decision can take effect while the model is stopped.

E. Make limits visible to the team

The proposed protocol gives each round a time limit and optional model-step and cost limits. Running time excludes paused intervals. The step limit counts model calls in the round's own context. The cost limit counts charged calls in that round and its descendants. Each call carries its owning round identifier; later rounds in a reused context do not inherit old spending. A renewed window resets the comparison baseline, not the usage history.

When a round pauses, its unresolved obligation remains visible to its parent. Running descendants pause with a link to the ancestor that caused the pause. On release, those descendants continue without receiving a fresh personal budget window merely because their ancestor resumed. The parent can tell the person that work awaits a decision while remaining able to receive new questions.

A limit is a point for control, not a proof of success or a universal spending cap. Repeated renewals can allow unbounded work. Cost observation can lag an active call. Therefore a soft threshold based on reported spending cannot enforce an exact hard cap. A study that requires a hard cap must reserve a stated upper charge before each call or stop admission when the remaining allowance cannot cover it. The method therefore reports used resources, pending work, and renewal count separately. Evaluation must count all workers against one experimental budget rather than giving every new helper a free allowance.

V. Worked Event Traces

We define three flawed baselines for the traces. **Activation-only close** ends a round whenever an activation ends, ignoring obligations. **Early release** tracks obligations but removes a child before accepting its result notice. **Unguarded close** uses notice-before-removal and reads obligations before context state, but then closes without checking revisions. Each baseline preserves other rules of the proposed protocol. Their distinct faults can therefore be derived from short event schedules.

A. A child result crosses a parent end

Consider a task context c that delegates a check to d. Its first activation returns a draft while d is still open. The following events are illustrative; their order, not their duration, is the result of interest.

TABLE II
The child-to-parent handoff keeps an obligation visible until the parent can incorporate the result.

Step	Event	Reason to stay open
1	c ends its draft activation	Child d remains
2	c accepts the result notice from d	Child and input overlap
3	c retires obligation d	Input or advancing activation
4	c reads and checks the result	Activation still advances
5	c ends with no further work	Round may close

A checker between steps 1 and 2 sees the open child. A checker after step 3 sees accepted input or the activation it started. If that activation already ended, its record includes a disposition for the notice. No delay estimate is required. Under early release, move step 3 before step 2. Between those steps a check sees zero children and no accepted input, so it can close on the draft. Activation-only close fails even earlier, at step 1. Both failures follow from their defined rules without any lost message.

For unguarded close, insert a user correction after the empty-state read at step 5 but before the close write. Admission joins the still-open round. The unchecked close then settles it without incorporating that accepted correction. With the proposed guard, the input either changes the checked revision and blocks close or follows the closed frontier and starts another round. This second race is distinct from the child handoff. Solving the first does not automatically solve the second.

B. Nested work, a pause, and a late extension

A lead task delegates a calculation to role B. B delegates a source check to role A, the same role as the lead but a different task context. B ends an activation while its source check remains open. The lead is then paused for a continuation

decision. Its running descendants pause because the ancestor stopped, not because each used its own limit.

Suppose, illustratively, the lead has run for 40 seconds of a 60-second window and is paused for 100 seconds by another control event. After release without renewal, 20 seconds remain: 60 minus 40. With a fresh allowed window, the comparison baseline becomes the usage at release and the lead gets 60 more running seconds. These are arithmetic examples, not observed timings.

The source check later returns. B integrates it and sends a completed result to the lead. The lead integrates both contributions and settles. The person then adds a missing requirement. A new round reuses the lead's task context. Its earlier result stays completed for its old frontier. Treating the whole context as permanently finished would reject useful follow-up; treating the old round as never finished would erase a meaningful record.

C. Progress without completion

Role A asks B to decide whether more evidence is needed. B asks a fresh instance of A for that decision. Each call returns promptly, so no call stack is blocked. Yet each new task creates one more unresolved child. The event stream is active and the team makes no semantic progress. This is livelock: activity continues without reaching the desired end.

A second case uses no new child tasks. A and B each wait for an answer from the other before offering a recommendation. Message delivery can be perfectly fair while neither produces the awaited answer. A third case sets another reminder on every wake. The task always has future work. Limits can expose these cases for a decision, but endless automatic renewal preserves them. Asynchronous delegation alone excludes none of them.

VI. Analytical Conditions

A. Safety of a child handoff

For normal delivery, assume a child obligation stays open until the parent accepts its result. Assume acceptance stays visible as pending input, an advancing activation, or a terminal record with its disposition. Assume the completion check reads obligations before context state. Then that check cannot close the parent solely by missing both this child and its unincorporated result.

The argument has two cases. If the obligation read precedes retirement, the obligation is open and the parent cannot close. If it follows retirement, acceptance preceded it. The subsequent context read therefore sees one of the three input states. Queued input or an active activation blocks close. A result with a recorded disposition need not block close. Cancellation is a separate terminal disposition, not evidence that the result was used. This proves a local handoff property. It does not prove an atomic snapshot across all obligation types or protect an unguarded check from future input.

B. The guarded close theorem

Theorem 1 (round close safety). Assume the work-record rules above, complete version changes, and an atomic comparison of both versions with close. If a round closes as completed at ordering point t, then it has no open obligation, advancing activation, pause, or accepted input without a use disposition at t. Every input admitted before t to that round belongs to its recorded frontier. Every task input admitted after t belongs to a later round.

Proof. Let t_w and t_v be the ordering points of the two reads, and let t be the successful close. A matching w means that no obligation mutation occurred between t_w and t. Otherwise that mutation would change w, which cannot return to its old value. The obligation block at t therefore equals the read block. The same argument with v preserves the context block from t_v to t. Both blocks passed the candidate test, so all their close conditions hold at t. The work-record invariant makes those blocks a complete account of admitted work. Input admission and close use the same ordering point, so admission either changes v before close or observes the settled round after it. The latter case must select a later round. The immutable close record preserves the old result. □

The two reads need not form one snapshot when taken. The successful comparison proves that each read value still holds at the shared commit point. This is why both version changes and the absence of counter reuse are required. A context-only guard misses a new child. An obligation-only guard misses a new correction. An unguarded close can miss either.

Lemma 2 (work-record preservation). Starting from valid empty records, each permitted transition preserves a witness for every accepted promise until it is disposed of. Registration creates an open obligation before dispatch. Acceptance adds input before retirement removes the obligation. Incorporation replaces pending input with a use record. Cancellation replaces it with an explicit abandonment record. Repeated delivery preserves the earlier acceptance, while a process stop changes no saved record. These cases cover the transitions that remove or add work. The lemma does not infer that a use record proves correct reasoning.

A completed result has only use dispositions. A failed or cancelled result may have abandonment dispositions and names the stop record. The theorem is about accounting safety. It cannot prove that an answer is true, that a dependency omitted by the model was noticed, or that an outside write was undone.

C. Eventual settlement

Assume that after some time no new external work is admitted to a chosen round. Assume its full future task and notice graph is finite and has no unresolved circular wait. Every enabled activation, notice delivery, and completion check eventually gets service. Every model or tool action ends, fails, or reaches a control boundary. Every decision and reminder eventually resolves. A finite number of budget renewals is allowed. The input and obligation accounting satisfies the safety conditions above.

Theorem 3 (conditional settlement). Under these assumptions, the round eventually settles. Consider the finite graph of remaining dependent events. An event with no unfinished prerequisite eventually runs by fairness. Remove that completed event and repeat. Since the graph is finite and has no depen-

dency cycle that prevents service, all obligations eventually resolve. Count duplicate messages as retries of an existing event, not fresh work. After the last work mutation, a fair close attempt has versions that remain fixed through commit. A final activation incorporates remaining input. A later completion check sees no work and closes the round. Failure and cancellation can shorten this process; the argument does not promise a successful answer.

Finite depth alone is insufficient because one task can create infinitely many siblings. Finite branching alone is insufficient because delegation can continue forever down a chain. Fair service alone is insufficient because a served task can always create another task. A finite cost budget bounds the action count only if every relevant action costs at least one fixed positive amount and the cap is enforced. Reminders, decisions, and unpriced model calls break that simple implication.

D. Responsiveness and resource bounds

Asynchronous handoff lets a parent end its current activation without waiting for a child. A separate direct context can thus be available while task contexts run. This is a structural opportunity for responsiveness. It is not a latency bound: shared model capacity, inbox backlog, and a long nonpreemptible call can still delay a reply.

For an illustrative scheduling guarantee, suppose a foreground service slot is offered after at most k background slices, including any current slice, each lasting at most s. A foreground reply needs one further slice, and admission overhead is at most a. With no earlier foreground backlog, its bound is $a + (k+1)s$. With $k = 3$, $s = 2$ seconds, and $a = 1$ second, the bound is $1 + 4 \times 2 = 9$ seconds. This is a bound for the defined scheduling model, not a measured system result. A reply needing several model calls requires a different service bound.

Budget accounting also needs a stated unit. A local step limit does not bound all descendant steps. A subtree cost total counts each charged call once within the round's descendant set. Nested task totals may overlap for policy checks, so adding all delegation totals would double-count spending in an experiment. Global evaluation cost must come from individual charged calls. Renewals should be visible events with a finite study allowance, not silent resets that advantage one policy.

VII. Model-Agent Evaluation Plan

The study tests result integration, responsiveness, and coordination cost at equal resources. Its unit is a complete task with scheduled follow-up messages. All conditions use the same model version, decoding settings, tools, starting facts, and artifact checks.

Use four workload families. Independent subtasks test parallel work with little coupling. Dependent subtasks require one child's evidence before another conclusion is valid. Revision tasks inject a change before, during, or just after candidate completion. Recursive tasks require bounded nested delegation and include deliberate circular-wait prompts. Include scheduled arrivals as external input, but stop new admissions at a declared time so round settlement has a defined target. Task correctness should be checkable with hidden facts, executable checks, or a fixed expert rubric.

Compare four policies. A single agent works serially in one context. A synchronous team waits for each child result. An asynchronous team returns receipts but closes on a terminal activation alone. The proposed team uses explicit obligations and a guarded frontier. Keep role prompts equal between team conditions, and give the single agent the same information. Vary task briefs only in a separate comparison.

Each instance gets one total token allowance, one call allowance, one cost allowance, and one elapsed-time deadline shared across all its workers. Report which cap ends each run. Use the same global concurrency ceiling. Coordination messages, summaries, retries, and result integration all consume the allowance. In one experimental block, keep the scheduler fixed to isolate completion policy. In a separate block, vary foreground service allocation to study responsiveness. This avoids crediting a completion rule for extra compute or better queue priority.

Before the main run, freeze one primary safety measure: the fraction of assigned tasks with at least one false close. Use the independent event log to reconstruct the obligation set and input membership at each close. The oracle is a separate check of the required end facts. Do not use the tested policy's own completed flag for that check. Retain tasks that time out in the denominator; report them as unsettled, not as safe successes.

Other measures are missed accepted input, task quality, and time to a useful response to an injected user request. False completion means closing a round with an obligation or input that belongs before its frontier still unresolved. A useful response must address the new request or give a truthful action-needed state; a generic acknowledgment does not qualify. Report settlement latency, successful-task rate, cost, peak active tasks, and the share of output that merely repeats earlier coordination text. Distinguish waiting for a person from runnable work denied service.

First enumerate deterministic schedules around result acceptance, obligation retirement, input reads, pauses, and admission. Then run paired model trials on the same tasks and arrival schedules. Choose the number of independent tasks using a pilot-based power analysis for the smallest effect that matters, and freeze it before the main comparison. Treat repeated seeds as nested observations within tasks. Use paired differences and 95% bootstrap confidence intervals, found by resampling whole tasks; do not count every message as an independent sample. Randomize policy order within each task and arrival schedule. Pair repeated seeds and keep all such seeds together when resampling a task. Publish both the number of tasks and runs. Mark additional comparisons as secondary and adjust their intervals or tests for multiple comparisons. A zero observed failure rate is an empirical count, not a proof of zero risk.

Measure response delay from the same injected input event to the first independently judged useful response. For timed-out runs, record a censored delay: the response was not seen before the deadline; do not compute latency only among successful runs. Report task success and delay together to show any tradeoff. Record model revision, provider settings,

each call's usage, and failed calls. Keep external service and scheduling failures visible in separate categories under a rule fixed before the run.

Ablate each obligation category, notice-before-close ordering, the revision guard, round reuse, and renewal limits. Each ablation targets a distinct error: lost results, missed decisions, forgotten reminders, false closure, or ambiguous late input. An ablation that saves cost by skipping required work should be reported as a quality or safety loss, not an efficiency gain. Publish all task texts, schedules, model settings, output checks, traces, and stopping rules. No outcome of this study is asserted here.

VIII. Executable Model and Proof Obligations

The repository contains a Python model in `papers/experiments/papers_01_05.py`, with the coordination experiment under paper ID `04`. Run all finite models from the repository root with:

```
devenv shell -- python3 papers/experiments/run.py \
  --seeds 30 --output .cache/e2e/papers.json
```

The initial handoff fixture executes all 24 permutations of acceptance, retirement, incorporation, and candidate close. Disabled actions have no effect and are not retried in that one pass. Two orders close in the first pass: acceptance, retirement, incorporation, close; and acceptance, incorporation, retirement, close. Both have a used result and no child obligation. The other 22 are not liveness failures: a required action was attempted before its precondition or before close became enabled. Each schedule also checks that repeat notice delivery adds no input. This is a complete enumeration of this four-action fixture, not of every task protocol trace.

Separate checks reject a stale input version and a stale obligation version. They preserve an earlier result when a later round is created. The pause arithmetic leaves 20 seconds without renewal and 60 with renewal. These are executed finite checks. They do not measure model reasoning, database atomicity, process recovery, or service response delay.

A. Separate reads and commit

The extended model in `papers/experiments/paper_04_extended.py` removes a key limit of the four-action fixture: candidate creation and commit are separate actions. It stores a live set of child IDs, accepted input IDs, used input IDs, and two versions. The close code sees only the saved read results and enabled version guards. An independent oracle also knows the promised result IDs. This oracle detects a dropped promise even if the tested close code sees no child and no input.

The base action set is accept, retire, incorporate, read obligations, read context, and commit. We enumerate all $6! = 720$ orders. A second family adds one correction; a third adds one new child. Each has $7! = 5,040$ orders. We run each family under five rules, for 54,000 schedule attempts and 374,400 checked action prefixes. The rules differ only in acceptance before retirement or which version guards they use. An action attempted before its precondition is a no-op. A closed round stays fixed, and later attempted actions are no-ops. Thus the search does not silently assign a late input to the old round.

TABLE III

Executed finite model results. Each cell is false closes / all committed closes. Each rule attempts 720 handoff orders and 5,040 orders in each other family. A commit is counted even when it is false. These counts are not failure rates for a real scheduler.

Rule	Handoff	Correction	New child
Full rule	0 / 3	0 / 12	0 / 3
Early retirement	6 / 12	14 / 40	14 / 20
No v guard	0 / 3	3 / 15	0 / 3
No w guard	0 / 3	0 / 12	7 / 10
Neither guard	0 / 3	3 / 15	7 / 10

The full rule has no false close in these bounds. It also has at least one valid close in every family, so a rule that simply refuses all completion cannot pass this check. Early retirement makes the required result disappear from the candidate's local view. Keeping both version guards cannot repair a promise that was already dropped before the reads. Removing only v admits a correction after the context read. Removing only w admits a child after the obligation read. The results isolate failures of work accounting and failures of atomic close.

Two further witness traces fix the race after both reads. Start with a used child result and its obligation retired. Read both blocks, admit a correction, then commit. The full rule refuses; removing v closes with an unused correction. Replace correction with new-child registration. The full rule again refuses; removing w closes with an open child. The output contains each action and full state for these witnesses, plus the first valid and invalid close for each explored rule.

The model fixes pause and activation flags at values that permit close. Incorporation stands for a completed activation. It has no real worker crash, owner change, cancellation, or database transaction. No-op actions are not retried, so the many orders without close say nothing about settlement under fair retries. Permutations are a finite state search, not independent samples from a workload. The model supports the local safety mechanism and its ablations; the theorem, implementation checks, and model-agent study have separate proof and evidence duties.

B. Further conformance targets

An event simulator needs round records, ordered inputs, activation records, obligation sets, and two revisions per context. Its commands are admit, start, record-disposition, register-obligation, accept-notice, retire-obligation, pause, resume, read-candidate, and close. Commands follow the protocol above. Close compares the candidate revisions and commits only if both match.

Start the handoff fixture with parent round p running, one terminal draft activation, empty input, and one child obligation d. The child has a terminal result and one notice ready to send. The enabled actions are accept-notice, retire-obligation, and the parent's candidate reads. In early release, permit retirement first. In the proposed protocol, retirement requires a saved acceptance. Enumerate every legal ordering of those actions, then start and end a parent activation when accepted input is available.

For the late-input fixture, start with no obligations and a terminal activation. Read a close candidate, admit correction x, then attempt close. The proposed close must fail because admission changes v. Process x and try again; the round can now settle. Reverse admission and a successful close to test the other branch: x must belong to the next round. These schedules define the round-membership check without model output.

The pause fixture uses the 40-of-60 timing trace above. Check both renewal choices and an inherited descendant pause. A deny decision must produce explicit cancellation dispositions. Removing its decision count alone cannot settle unfinished work.

TABLE IV
CONFORMANCE CHECKS FOR THE DEFINED PROTOCOL. THIS IS THE FULL CONFORMANCE TARGET. THE TEXT ABOVE IDENTIFIES THE SUBSET CHECKED BY THE FINITE MODEL; THE TABLE ALONE DOES NOT REPORT A TEST RESULT.

Fixture change	Required observation
Repeat a notice ID	One accepted input position
Create a child after a candidate read	Changed w rejects close
Admit correction before close	Changed v rejects close
Admit extension after close	New round; old result unchanged
Deliver notice to settled target	History entry; no new activation
Resolve a reminder	Notice accepted before obligation retires

A simulator should reject any state that loses an input disposition, removes an obligation without acceptance or explicit cancellation, or modifies a settled round's result. Bounded exploration can find a counterexample but cannot prove the protocol for unbounded task trees. The proof obligations remain atomic revision comparison, complete mutation coverage, faithful notice identity, and preservation of accepted input until disposition. Those requirements must be checked for any implementation used in the model experiment.

The study also has limits. A model may record use of an input while failing to apply it correctly; task-quality checks must detect that gap. Context reuse may preserve useful evidence or carry old assumptions into another round. Waiting for a human decision has no finite upper bound without a separate deadline policy. Shared service capacity may make a logically independent direct context slow. The protocol supplies no general freedom from deadlock and no guarantee of semantic progress. Its claims concern accounting and settlement under the stated assumptions.

IX. CONCLUSION

Long-lived agent coordination needs an explicit boundary between a worker, its context, and one round of assigned work. Open obligations and accepted input explain why an ended activation is not enough to close a task. Keeping a result visible during handoff and recording a completion frontier give precise conditions for safe settlement. Eventual settlement additionally needs finite work, resolved waits, and fair service. The finite model checks specified event orders. A separate proposed model-agent study would test responsiveness and result quality at matched models and budgets. Asynchronous calls create room for useful coordination; they do not, by themselves, prove that a team will finish.

REFERENCES

[1] C. Hewitt, P. Bishop, and R. Steiger, "A Universal Modular ACTOR Formalism for Artificial Intelligence," in *Proceedings of the 3rd International Joint Conference on Artificial Intelligence*, 1973, pp. 235–245. [Online]. Available: https://www.ijcai.org/Proceedings/73/Papers/027B.pdf

[2] E. W. Dijkstra and C. S. Scholten, "Termination detection for diffusing computations," *Information Processing Letters*, vol. 11, no. 1, pp. 1–4, 1980, doi: 10.1016/0020-0190(80)90021-6.

[3] K. M. Chandy and L. Lamport, "Distributed Snapshots: Determining Global States of Distributed Systems," *ACM Transactions on Computer Systems*, vol. 3, no. 1, pp. 63–75, 1985, doi: 10.1145/214451.214456.

[4] Q. Wu *et al.*, "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation." [Online]. Available: https://arxiv.org/abs/2308.08155v2

[5] G. Li, H. A. A. K. Hammoud, H. Itani, D. Khizbullin, and B. Ghanem, "CAMEL: Communicative Agents for `Mind' Exploration of Large Language Model Society." [Online]. Available: https://arxiv.org/abs/2303.17760v2

[6] F. Mattern, "Algorithms for distributed termination detection," *Distributed Computing*, vol. 2, no. 3, pp. 161–175, 1987, doi: 10.1007/BF01782776.

[7] D. Gao *et al.*, "AgentScope: A Flexible yet Robust Multi-Agent Platform." [Online]. Available: https://arxiv.org/abs/2402.14034v2

[8] M. Cemri *et al.*, "Why Do Multi-Agent LLM Systems Fail?." [Online]. Available: https://arxiv.org/abs/2503.13657v3

[9] M. P. Herlihy and J. M. Wing, "Linearizability: A Correctness Condition for Concurrent Objects," *ACM Transactions on Programming Languages and Systems*, vol. 12, no. 3, pp. 463–492, 1990, doi: 10.1145/78969.78972.